WEEK-7

🐚 Shell Programming Basics

1. What is a Shell?

- A **command interpreter** between user and OS kernel.
- Executes commands, scripts, and automates tasks.

2. Types of Shell in Linux

- **Bourne Shell (sh)** – original Unix shell.
- **Bash (Bourne Again Shell)** – most common, feature-rich.
- **C Shell (csh)** – syntax similar to C language.
- **Korn Shell (ksh)** – combines features of sh and csh.
- **Z Shell (zsh)** – advanced, customizable.

Check your shell: **echo $SHELL**

🔹 What is Shell Programming?

Shell Programming means:

👉 Writing multiple Linux commands inside a file
👉 Saving it as a script
👉 Executing it like a program

Instead of typing commands again and again, we automate them.

📜 Basic Shell Script

📌 Simple Structure of Any Shell Script

1. Shebang
2. Variables
3. Input
4. Processing (if/loop)
5. Output

🔹 Step 1: Create Script File

```
nano myscript.sh
```

🔹 Step 2: Add Shebang (Very Important)

At top of file:

```
#!/bin/bash
```

🔹 What is Shebang (Hashbang)?

`#!` tells system:

👉 Which interpreter to use
👉 Here → Use bash shell

Without this, the script may not run properly.

🔹 Step 3: Write Commands

Example:

```
#!/bin/bash
echo "Hello Students"
echo "Welcome to Shell Programming"
```

🔹 Step 4: Give Permission

```
chmod +x myscript.sh
```

🔹 Step 5: Execute

```
./myscript.sh
```

Output:

Hello Students
Welcome to Shell Programming

📥 Input & Output in Shell

- 🔹 **Output → echo**
- 🔹 **Input → read**

Example:

```
#!/bin/bash
echo "Enter your name:"
read name
echo "Your name is $name"
```

Run:

Enter your name:
Ravi
Your name is Ravi

👉 `$name` is variable

Understanding Bash Operators

This section provides an overview of operators used in Bash scripting, including comparison, string, arithmetic, logical, and file test operators.

Comparison Operators

- `-eq` : Equal to
- `-ne` : Not equal to
- `-lt` : Less than
- `-le` : Less than or equal to
- `-gt` : Greater than
- `-ge` : Greater than or equal to

String Comparison Operators

- `=` : Equal to
- `!=` : Not equal to
- `<` : Less than, in ASCII alphabetical order
- `>` : Greater than, in ASCII alphabetical order

Arithmetic Operators

- `+`: Addition
- `-`: Subtraction
- `*`: Multiplication
- `/`: Division
- `%`: Modulus (remainder of division)
- For exponentiation, use external tools like `bc` or `awk`.

Logical Operators

- `&&`: Logical AND
- `||`: Logical OR
- `!`: Logical NOT

File Test Operators

- `-e`: Checks if a file exists
- `-d`: Checks if a directory exists
- `-f`: Checks if a file is a regular file
- `-s`: Checks if a file is not empty

🔀 Decision Making in Shell

`if` Statement

Bash Copy

```bash
if [ condition ]
then
   commands
else
   commands
fi
```

Example (check file existence):

Bash Copy

```bash
#!/bin/bash
if [ -f myfile.txt ]
then
   echo "File exists"
else
   echo "File not found"
fi
```

1. For Loop

- **Syntax:**

Bash Copy

```bash
for variable in list
do
    commands
done
```

- **Example:**

Bash Copy

```bash
for i in 1 2 3 4 5
do
    echo "Number: $i"
done
```

2. While Loop

- **Syntax:**

Bash Copy

```bash
while [ condition ]
do
    commands
done
```

- **Example:**

Bash Copy

```bash
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count+1))
done
```

3. Until Loop

- **Syntax:**

Bash Copy

```bash
until [ condition ]
do
    commands
done
```

- **Example:**

Bash Copy

```bash
num=5
until [ $num -lt 1 ]
do
    echo "Countdown: $num"
    num=$((num-1))
done
```

File Operations

Create & Write

Bash Copy

```bash
echo "This is a test" > file.txt
```

Append

Bash Copy

```bash
echo "Another line" >> file.txt
```

Read

Bash Copy

```bash
while read line
do
   echo $line
done < file.txt
```

Delete

Bash Copy

```bash
rm file.txt
```

Check File Existence

- **Syntax:**

Bash Copy

```bash
if [ -f filename ]
then
   echo "File exists"
else
   echo "File not found"
fi
```

📝 Short Definitions

✔ Shell → Command interpreter
✔ Shell script → File containing Linux commands
✔ Shebang → Specifies interpreter
✔ read → Takes input
✔ if → Decision making
✔ loop → Repetition

PRACTICE PROGRAMS

📜 Script: Biggest of Three Numbers

Bash ^ Copy

```bash
#!/bin/bash
# Script to find the biggest of three numbers

echo "Enter three numbers:"
read a b c

if [ $a -ge $b ] && [ $a -ge $c ]
then
    echo "The biggest number is: $a"
elif [ $b -ge $a ] && [ $b -ge $c ]
then
    echo "The biggest number is: $b"
else
    echo "The biggest number is: $c"
fi
```

📄 Explanation

- `read a b c` → takes three numbers as input.
- `if [ $a -ge $b ] && [ $a -ge $c ]` → checks if `a` is greater than or equal to both `b` and `c`.
- `elif` → checks if `b` is greater than or equal to both `a` and `c`.
- `else` → if neither condition is true, then `c` must be the largest.

📜 Script: Iterations in Shell

Bash ^ Copy

```bash
#!/bin/bash
# Script to illustrate different types of iterations

echo "=== FOR LOOP ==="
for i in {1..5}
do
    echo "Number: $i"
done

echo "=== WHILE LOOP ==="
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count+1))
done

echo "=== UNTIL LOOP ==="
num=5
until [ $num -lt 1 ]
do
    echo "Countdown: $num"
    num=$((num-1))
done
```

📃 Explanation

- **For loop:** Iterates over a fixed range `{1..5}`.
- **While loop:** Runs as long as the condition `[ $count -le 5 ]` is true.
- **Until loop:** Runs until the condition `[ $num -lt 1 ]` becomes true (i.e., stops when num < 1).

📜 Script: String Operations

Bash ^ Copy

```bash
#!/bin/bash
# Script to perform string operations

echo "Enter first string:"
read str1
echo "Enter second string:"
read str2

# 1. Length of strings
echo "Length of first string: ${#str1}"
echo "Length of second string: ${#str2}"

# 2. Concatenation
concat="$str1$str2"
echo "Concatenated string: $concat"

# 3. Comparison
if [ "$str1" = "$str2" ]
then
    echo "Strings are equal"
else
    echo "Strings are not equal"
fi

# 4. Substring extraction (first 3 characters of str1)
echo "Substring of first string (first 3 chars): ${str1:0:3}"

# 5. Convert to uppercase and lowercase
echo "First string in uppercase: ${str1^^}"
echo "Second string in lowercase: ${str2,,}"
```

📄 Explanation

- `${#str1}` → gives the length of the string.
- `"$str1$str2"` → concatenates two strings.
- `[ "$str1" = "$str2" ]` → compares strings for equality.
- `${str1:0:3}` → extracts substring starting at index 0 with length 3.
- `${str1^^}` → converts to uppercase.
- `${str2,,}` → converts to lowercase.

📜 Script: Count Lines, Words, and Characters

Bash Copy

```bash
#!/bin/bash
# Script to count lines, words, and characters in a file

echo "Enter the filename:"
read filename

if [ -f $filename ]
then
    echo "File '$filename' exists."
    echo "Counting..."
    lines=$(wc -l < $filename)
    words=$(wc -w < $filename)
    chars=$(wc -c < $filename)

    echo "Number of lines: $lines"
    echo "Number of words: $words"
    echo "Number of characters: $chars"
else
    echo "File '$filename' not found."
fi
```

📜 Script: File Operations

Bash

```bash
#!/bin/bash
# Script to perform file operations

echo "Enter filename:"
read filename

# 1. Create a file
touch $filename
echo "File '$filename' created."

# 2. Write to the file
echo "Enter text to write into the file:"
read text
echo "$text" > $filename
echo "Text written to '$filename'."

# 3. Display file contents
echo "Contents of '$filename':"
cat $filename

# 4. Append to the file
echo "Enter text to append to the file:"
read append_text
echo "$append_text" >> $filename
echo "Text appended to '$filename'."

# 5. Display updated contents
echo "Updated contents of '$filename':"
cat $filename

# 6. Rename the file
echo "Enter new filename to rename:"
read newname
mv $filename $newname
echo "File renamed to '$newname'."

# 7. Delete the file
echo "Do you want to delete '$newname'? (y/n)"
read choice
if [ "$choice" = "y" ]
then
    rm $newname
    echo "File '$newname' deleted."
else
    echo "File '$newname' not deleted."
fi
```

https://copilot.microsoft.com/shares/ZJ5R4JEZ1go8AMqLfBXSK

Script: Menu-Driven File Operations

```
#!/bin/bash
# Menu-driven script to perform file operations

filename="demo.txt"

while true
do
    echo "==============================="
    echo " File Operations Menu "
    echo "==============================="
    echo "1. Create and Write to File"
    echo "2. Append to File"
    echo "3. Read File"
    echo "4. Count Lines in File"
    echo "5. Delete File"
    echo "6. Exit"
    echo "Enter your choice:"
    read choice

    case $choice in
        1)
            echo "Enter text to write:"
            read text
            echo "$text" > $filename
            echo "File '$filename' created/written."
            ;;
        2)
            echo "Enter text to append:"
```

```
    read text
    echo "$text" >> $filename
    echo "Text appended to '$filename'."
    ;;
3)
    if [ -f $filename ]
    then
        echo "Contents of '$filename':"
        cat $filename
    else
        echo "File does not exist."
    fi
    ;;
4)
    if [ -f $filename ]
    then
        lines=$(wc -l < $filename)
        echo "Number of lines in '$filename': $lines"
    else
        echo "File does not exist."
    fi
    ;;
5)
    if [ -f $filename ]
    then
        rm $filename
        echo "File '$filename' deleted."
    else
        echo "File does not exist."
    fi
    ;;
6)
```

```
        echo "Exiting program..."
        break
        ;;
    *)
        echo "Invalid choice. Please try again."
        ;;
  esac
done
```

CASE STATEMENT SYNTAX

Bash Copy

```bash
case $choice in
    1)
        # commands for option 1
        ;;
    2)
        # commands for option 2
        ;;
    *)
        # default case
        ;;
esac
```